

1. Comandos Úteis (CLI)

- **Spring Boot (Backend):**
 - Execução rápida: `mvnw spring-boot:run`
 - Compilação e empacotamento: `mvnw clean install`
- **React / Node.js (Frontend):**
 - Criar projeto: `npx create-react-app nome`
 - Instalar dependências: `npm install`
 - Iniciar servidor de desenvolvimento: `npm start` / `npx run dev`
 - Roteamento: `npm install react-router-dom`

2. Backend (Spring Boot / Java)

Dependências e Módulos Comuns

- Web (`spring-boot-starter-web`), JPA (`spring-boot-starter-data-jpa`), Driver do Banco de Dados (MySQL), Validation, Security (JWT/BCrypt), Mail, Thymeleaf e Lombok.

Anotações e Mapeamentos Principais

- **Controladores (@RestController, @RequestMapping):**
 - `@PostMapping`: Criação de recursos.
 - `@PutMapping`: Atualização de recursos.
 - `@GetMapping`: Busca de recursos.
 - `@DeleteMapping`: Remoção de recursos.
 - `@CrossOrigin`: Libera requisições entre origens distintas diretamente na classe/método.
 - `@PathVariable` e `@PathParam` (`@RequestParam`): Manipulação de parâmetros de URL.
- **Persistência (@Entity, JPA, Repositories):**
 - Estruturas baseadas em `JpaRepository<Entidade, Long>`.
 - Criação de métodos customizados de consulta por convenção (ex: `findByEmail`) ou com consulta JPQL usando `@Query` e `@Param`.
- **Segurança e JWT:**
 - Uso de `SecurityFilterChain` sem estado (`SessionCreationPolicy.STATELESS`), desativando CSRF e configurando origens permitidas via CORS.
 - Filtro customizado `JwtFiltroAutenticacao` herdando de `OncePerRequestFilter` para validar tokens via header `Authorization: Bearer <token>`.
 - Utilitário `JwtService` para criação, extração e validação de tokens JWT via `io.jsonwebtoken`.
 - Implementação de `UserDetails` na entidade `Person` com criptografia de senha via `BCryptPasswordEncoder`.

Tratamento de Exceções e Serviços Auxiliares

- `@RestControllerAdvice` e `@ExceptionHandler` para padronizar retornos de erro (404 Not Found, 422 Unprocessable Entity, 400 Bad Request para validação de campos).

- **EmailService**: Envio de e-mails simples via **JavaMailSender** e e-mails formatados em HTML usando templates do Thymeleaf (**TemplateEngine**).
- **AuctionWebSocketController**: Implementação de WebSocket com **@MessageMapping** e **@SendTo** para atualizações de lances em tempo real.

3. Diagrama de Classes e Regras do Sistema de Leilão (UML)

- **Entidades Mapeadas**: **Pessoa**, **Perfil**, **Categoria**, **Leilao**, **Imagem**, **Lance**, **Feedback**, **Pagamento**.
- **Enums**:
 - **StatusLeilao**: ABERTO, ENCERRADO, CANCELADO, EM_ANALISE.
 - **TipoPerfil**: ADMIN, COMPRADOR, VENDEDOR.
- **Relacionamentos Principais**:
 - Relação N:N entre **Pessoa** e **Perfil** através da classe associativa **PessoaPerfil**.
 - **Pessoa** publica **Leilao**, realiza **Lance** e envia/recebe **Feedback**.
 - **Leilao** possui **Categoria**, múltiplas **Imagens** e múltiplos **Lances**.

4. Frontend (React / JavaScript)

Conceitos Essenciais

- **Componentização**: Declaração de componentes funcionais em JS/JSX.
- **Props**: Passagem de parâmetros do componente pai para o filho.
- **Hooks Principais**:
 - **useState**: Gerenciamento do estado local da aplicação.
 - **useEffect**: Execução de efeitos colaterais e sincronização do ciclo de vida.
 - **useNavigate**: Navegação programática de rotas.
 - **useRef**: Acesso direto a elementos ou persistência de valores (ex: componentes **Toast**).

Comunicação com API (Axios)

- Instância customizada do **axios** configurada com a URL base da API.
- **Interceptors**: Adição automática do token de autenticação JWT capturado do **localStorage** no cabeçalho das requisições.
- **Classes de Serviço (BaseService)**: Abstração de chamadas CRUD (**list**, **insert**, **update**, **delete**) reutilizadas por serviços específicos como **CategoryService**, **PerfilService** e **AuctionService**.

Biblioteca de Interface (PrimeReact)

- Utilização de componentes visuais avançados da PrimeReact: **DataTable** e **Column** para exibições e listagens, **Dialog** para telas modais de cadastro/edição, **Toast** e **ConfirmDialog** para notificações e confirmações, **Menubar** para navegação e temas como **lara-light-indigo**.

Internacionalização e Rotas Protegidas

- Configuração do **i18next** para suporte a múltiplos idiomas (PT/EN).
- **RotaPrivadaLayout**: Proteção de rotas do sistema verificando credenciais no **localStorage** e redirecionando usuários não autenticados para a tela de login (**/login**).

